



INSTITUTO POLITÉCNICO DE LISBOA
INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

Licenciatura em Engenharia Informática e de Computadores

NatureProtector
Descrição da Organização do Repositório

Miguel Alves n.º 51650, e-mail: a51650@alunos.isel.pt
Gabriel Mano n.º 51951, e-mail: a51951@alunos.isel.pt

Orientadores: Artur Ferreira
Nuno Leite

Projeto e Seminário

31 de maio de 2026

Índice

1	Identificação do Projeto	1
1.1	Projeto e contexto acadêmico	1
1.2	Finalidade do documento	1
1.3	Âmbito e limites	1
2	Acesso ao Repositório	2
2.1	Localização	2
2.2	Clonagem do repositório	2
2.3	Setup local	2
3	Visão Geral da Organização	3
3.1	Estrutura de alto nível	3
3.2	Principais blocos do repositório	3
3.3	Separação entre código, documentação e evidência	4
3.4	Artefactos gerados	4
4	Estrutura de Pastas	4
4.1	Raiz do repositório	4
4.2	Código fonte em src/	4
4.3	Projetos de teste em tests/	5
4.4	Interface web em webUI/	6
4.5	Documentação em docs/	6
4.6	Infraestrutura em infra/	7
4.7	Scripts em scripts/	7
4.8	Dados e manifestos em data/	8
5	Execução, Validação e Evidência	8
5.1	Requisitos gerais	8
5.2	Setup local	8
5.3	Build e testes .NET	9
5.4	Interface web	9
5.5	Infraestrutura local	9
5.6	Runtime de desenvolvimento	9
5.7	Execução de cenários e evidência	10
5.8	Coverage	10

1 Identificação do Projeto

1.1 Projeto e contexto académico

O presente documento descreve a organização do repositório do projeto **NatureProtector**, desenvolvido no contexto da unidade curricular de Projeto e Seminário da Licenciatura em Engenharia Informática e de Computadores do Instituto Superior de Engenharia de Lisboa.

O NatureProtector é um sistema de apoio à prevenção e monitorização operacional de risco de incêndio rural. O repositório integra o código fonte, os testes, a infraestrutura local, a interface web, os scripts de execução, os dados de suporte, a documentação técnica, a evidência recolhida durante a validação e os materiais de entrega associados ao projeto.

Campo	Descrição
Projeto	NatureProtector
Instituição	Instituto Superior de Engenharia de Lisboa
Curso	Licenciatura em Engenharia Informática e de Computadores
Unidade curricular	Projeto e Seminário
Autores	Miguel Alves; Gabriel Mano
Orientadores	Artur Ferreira; Nuno Leite
Repositório	GitHub público do projeto

1.2 Finalidade do documento

A finalidade deste documento é apresentar a organização do repositório de forma objetiva, permitindo a um avaliador, orientador ou novo elemento da equipa localizar rapidamente os principais artefactos do projeto.

O documento identifica:

- a localização pública do repositório;
- a estrutura de alto nível das pastas;
- os projetos de código fonte e respetivas responsabilidades;
- os projetos de teste;
- a interface web;
- os dados, manifestos, scripts, infraestrutura e documentação;
- os comandos principais de validação;
- os artefactos gerados que não devem ser confundidos com código fonte.

1.3 Âmbito e limites

Este documento não substitui o relatório técnico do projeto, a documentação de arquitetura nem o guia completo de setup. A sua função é descrever a organização do repositório e indicar onde se encontram os artefactos relevantes.

Assim, ficam fora do âmbito deste documento:

- a descrição científica completa da fórmula de risco;
- a fundamentação detalhada dos índices FWI e KBDI;
- a discussão metodológica completa da V1;
- a validação científica final do modelo;

- o manual exaustivo de utilização da aplicação.

A referência principal para preparar o ambiente local é o ficheiro `docs/local-baseline-setup.md`. Este documento apenas indica essa referência e resume os comandos de validação mais relevantes.

2 Acesso ao Repositório

2.1 Localização

O repositório do projeto encontra-se disponível publicamente no GitHub:

<https://github.com/MAlves4018/NatureProtector.git>

Por se tratar de um repositório público, a consulta do código fonte, documentação e artefactos versionados não requer permissões adicionais de leitura.

2.2 Clonagem do repositório

A obtenção local do repositório pode ser feita com:

```
git clone https://github.com/MAlves4018/NatureProtector.git
cd NatureProtector
```

Após a clonagem, recomenda-se confirmar a branch e o commit em uso:

```
git branch --show-current
git log -1 --oneline
git status --short
```

Estas informações permitem associar qualquer evidência de execução, teste ou demonstração a uma versão concreta do repositório.

2.3 Setup local

O setup local não é duplicado neste documento. A referência de setup é:

`docs/local-baseline-setup.md`

Esse ficheiro deve ser lido antes de tentar executar a baseline local. Em termos gerais, o repositório depende de um ambiente com .NET SDK, Docker Desktop, PowerShell e Node.js/npm para executar a solução, a infraestrutura local e a interface web.

3 Visão Geral da Organização

3.1 Estrutura de alto nível

O repositório está organizado por responsabilidades técnicas. A raiz contém a solução .NET, a configuração global, a infraestrutura local, a interface web, os dados de suporte, os scripts, os testes e a documentação.

Uma vista simplificada da raiz é a seguinte:

```
NatureProtector/  
|-- data/  
|-- docs/  
|-- infra/  
|-- scripts/  
|-- src/  
|-- tests/  
|-- webUI/  
|-- docker-compose.yml  
|-- NatureProtector.sln  
|-- NuGet.Config  
|-- coverage.runsettings  
'-- README.md
```

Esta organização separa o código executável dos testes, dos dados, dos scripts de automação, da documentação e dos artefactos de infraestrutura.

3.2 Principais blocos do repositório

Pasta/Ficheiro	Função principal
src/	Código fonte dos projetos .NET, incluindo domínio, hosts, API, persistência e infraestrutura.
tests/	Testes unitários, testes de integração e testes por módulo.
webUI/	Interface web usada para monitorização, execução de cenários, diagnósticos e evidência.
docs/	Documentação técnica, arquitetura, contratos, evidência, relatório, poster e materiais auxiliares.
infra/	Configuração da infraestrutura local, incluindo PostgreSQL, Grafana e scripts de infraestrutura.
scripts/	Scripts de setup, execução, cenários, evidência, documentação, testes e apoio ao desenvolvimento.
data/	Dados de suporte, baseline, manifestos de datasets e manifestos de cenários.
docker-compose.yml	Definição dos serviços locais usados pela baseline.
NatureProtector.sln	Solução .NET principal.
coverage.runsettings	Configuração usada para recolha de cobertura de testes.

3.3 Separação entre código, documentação e evidência

O repositório não contém apenas código fonte. Contém também documentos de arquitetura, mapas de implementação, contratos, dados, scripts de execução e evidência runtime. Esta separação permite consultar não só a implementação, mas também a forma como a implementação foi validada.

A documentação encontra-se sobretudo em `docs/`. A evidência de execução e validação é guardada em `docs/evidence/`, enquanto scripts reproduzíveis de apoio se encontram em `scripts/`. Esta organização facilita a auditoria do projeto e reduz a dependência de explicações orais.

3.4 Artefactos gerados

Algumas pastas podem existir localmente por serem geradas durante build, testes, coverage, execução da UI ou execução de scripts. Exemplos comuns são:

- `bin/` e `obj/`;
- `.testbin/`;
- `TestResults/`;
- `coveragereport_core/`;
- `node_modules/`;
- `__pycache__/`;
- ficheiros auxiliares gerados por LaTeX, como `.aux`, `.log`, `.fls` ou `.toc`.

Estes artefactos não devem ser confundidos com a estrutura fonte do repositório e devem estar cobertos por `.gitignore`, salvo quando a equipa decidir versionar explicitamente algum output final.

4 Estrutura de Pastas

Esta secção descreve as principais pastas do repositório e a responsabilidade de cada uma. A listagem completa do repositório é extensa; por isso, apresenta-se uma descrição resumida e orientada à navegação.

4.1 Raiz do repositório

A raiz reúne os ficheiros de entrada do projeto e as principais pastas de trabalho.

Elemento	Responsabilidade
<code>README.md</code>	Entrada principal de leitura do repositório.
<code>NatureProtector.sln</code>	Solução .NET principal.
<code>Directory.Build.props</code>	Configuração comum de build para os projetos .NET.
<code>NuGet.Config</code>	Configuração de fontes NuGet usadas pela solução.
<code>coverage.runsettings</code>	Configuração de recolha de cobertura.
<code>docker-compose.yml</code>	Definição dos serviços locais de infraestrutura.

4.2 Código fonte em `src/`

A pasta `src/` contém os projetos .NET da solução. A organização segue uma separação por responsabilidades: domínio, prevenção, simulador, API, persistência, infraestrutura e contratos

partilhados.

Projeto	Responsabilidade
<code>src/NatureProtector.Core</code>	Conceitos base do domínio, incluindo áreas, sensores, leituras, cenários, risco e objetos de valor partilhados.
<code>src/NatureProtector.Prevention</code>	Domínio de prevenção, incluindo normalização, elegibilidade, componentes de risco, FWI, KBDI, contexto territorial e scoring.
<code>src/NatureProtector.Prevention.Host</code>	Serviço de runtime que consome eventos, processa leituras, regista tentativas, atualiza estados diários, risk assessments e projeções.
<code>src/NatureProtector.Simulator.Host</code>	Geração de cenários, truth snapshots, observações locais, perfis de degradação e publicação de leituras simuladas.
<code>src/NatureProtector.Shared</code>	Contratos partilhados, envelope de eventos, payloads, configuração, RabbitMQ topology e observabilidade comum.
<code>src/NatureProtector.Infrastructure.Postgres</code>	DbContext, migrations, bootstrap, entidades de persistência e schemas de controlo, pipeline e projeção.
<code>src/NatureProtector.Infrastructure.Influx</code>	Serviços e opções para escrita opcional de telemetria temporal em InfluxDB.
<code>src/NatureProtector.Backoffice.Api</code>	API de leitura e controlo para a UI, incluindo endpoints de áreas, runtime summary, diagnostics, runs e reset.
<code>src/NatureProtector.Postgres.Bootstrap</code>	Aplicação de inicialização e carregamento do plano de controlo em PostgreSQL.
<code>src/NatureProtector.AppHost</code>	Orquestração local/experimental, quando aplicável ao ambiente de desenvolvimento.

4.3 Projetos de teste em tests/

A pasta `tests/` espelha os principais módulos de `src/`. A intenção é validar o domínio, a pipeline, a infraestrutura, os contratos, a API e os cenários de integração.

Projeto de testes	O que valida
<code>tests/NatureProtector.Core.Tests</code>	Entidades e conceitos base do domínio.
<code>tests/NatureProtector.Prevention.Tests</code>	Normalização, elegibilidade, scoring, FWI, KBDI, classes de risco, contexto territorial e regras da prevenção.
<code>tests/NatureProtector.Prevention.Host.Tests</code>	Pipeline de processamento, inbox, tentativas, persistência, projeções e políticas operacionais.
<code>tests/NatureProtector.Simulator.Host.Tests</code>	Geração de leituras, cenários, perfis de degradação e comportamento do simulador.

Projeto de testes	O que valida
tests/NatureProtector.Backoffice.Api.Tests	Controllers, serviços de API, contratos de resposta, runtime diagnostics e endpoints de controle.
tests/NatureProtector.Shared.Tests	Contratos compartilhados, serialização, envelope de eventos e configuração comum.
tests/NatureProtector.Infrastructure.Influx.Tests	Escrita Influx, opções, serviços safe/no-op e configuração de observabilidade temporal.
tests/NatureProtector.IntegrationTests	Fluxos integrados e cenários que atravessam mais do que um módulo.

Pastas como TestResults/, bin/, obj/ ou .testbin/ são geradas durante a execução dos testes e não devem ser tratadas como código fonte.

4.4 Interface web em webUI/

A pasta webUI/ contém a interface web, usada para observar o estado operacional, executar cenários, consultar diagnósticos, visualizar mapas, comparar runs e exportar evidência.

Pasta/Ficheiro	Função
webUI/src/app/components/ webUI/src/app/services/	Componentes e páginas principais da interface. Cliente HTTP e serviços de comunicação com a Backoffice API.
webUI/src/app/types/	Tipos usados pela UI para representar respostas da API.
webUI/src/app/utils/	Funções auxiliares de formatação e suporte à interface.
webUI/public/	Recursos públicos, imagens, links de dashboards e ficheiros estáticos.
webUI/package.json	Dependências e scripts npm da UI.
webUI/vite.config.ts	Configuração Vite, incluindo proxy para a API local.

A UI não deve recalcular risco no frontend. Os scores, diagnósticos e estados operacionais são obtidos através da Backoffice API.

4.5 Documentação em docs/

A pasta docs/ reúne documentação técnica, arquitetura, contratos, evidência e materiais de entrega. É uma das áreas mais importantes do repositório, porque liga o comportamento executável à explicação técnica do projeto.

Pasta/Ficheiro	Conteúdo
docs/NatureProtector-V1-overview.md	Síntese da V1, estado técnico e enquadramento operacional.
docs/architecture/	Arquitetura, diagramas, guias de runtime, Grafana, PostgreSQL e orquestrador de cenários.

Pasta/Ficheiro	Conteúdo
docs/contracts/	Vocabulário, catálogo de eventos e contratos conceptuais.
docs/evidence/	Evidência de execução, logs, runs, outputs de validação, SQL e resultados de testes.
docs/implementation/	Planeamento de implementação, tarefas, fases e mapas entre proposta e implementação.
docs/planning/	Planeamento complementar e matrizes de alinhamento entre pesquisa e implementação.
docs/setup/	Documentação auxiliar de setup, quando aplicável.
docs/report/	Relatório LaTeX, poster e documentos de entrega relacionados.
docs/structurizr/, docs/doxygen/, docs/docfx/	Documentação e modelos técnicos gerados ou preparados por ferramentas.

A referência de setup indicada para a baseline local é docs/local-baseline-setup.md.

4.6 Infraestrutura em infra/

A pasta infra/ contém configuração para serviços locais e recursos de suporte à demonstração.

Pasta	Conteúdo
infra/grafana/	Dashboards, provisioning, datasources e configuração de visualização.
infra/postgres/	Scripts de inicialização e configuração PostgreSQL.
infra/scripts/	Scripts auxiliares de infraestrutura, incluindo arranque de serviços locais.

A infraestrutura local inclui serviços como PostgreSQL, RabbitMQ, InfluxDB e Grafana, definidos ou apoiados por docker-compose.yml e scripts associados.

4.7 Scripts em scripts/

A pasta scripts/ contém automação para setup, desenvolvimento, cenários, documentação, evidência, testes e apoio à infraestrutura.

Pasta	Função
scripts/setup/	Scripts de verificação e preparação do ambiente local.
scripts/dev/	Scripts para arranque da runtime de desenvolvimento.
scripts/scenarios/	Exemplos e scripts para execução de cenários.
scripts/evidence/	Scripts para recolha de evidência, smoke tests e comparação de runs.
scripts/tests/	Scripts de apoio à execução de testes e coverage.
scripts/data/	Scripts de preparação, geração ou curadoria de dados e manifestos.
scripts/docs/	Scripts de apoio à documentação.

Pasta	Função
scripts/postgres/, scripts/influx/	Scripts específicos de suporte a serviços de persistência e observabilidade.

4.8 Dados e manifestos em data/

A pasta data/ contém dados de suporte à baseline e manifestos usados por scripts e cenários.

Pasta/Ficheiro	Conteúdo
data/baseline/areas/proenca-a-nova/	Dados de referência da área, incluindo geometria, células, atributos, histórico, observações e referências meteorológicas.
data/manifests/datasets/	Manifestos de datasets, incluindo o plano de dados de Proença-a-Nova.
data/manifests/scenarios/	Manifestos de cenários, incluindo cenários base, high-risk e degraded-pipeline.
data/runtime/	Dados criados durante execução local, como artefactos runtime de serviços.

Os dados em data/runtime/ dependem da execução local e não devem ser interpretados como configuração estável do projeto.

5 Execução, Validação e Evidência

Esta secção resume os pontos principais para validação do repositório. O procedimento completo de setup deve ser consultado em docs/local-baseline-setup.md. Este documento não substitui esse guia.

5.1 Requisitos gerais

A execução local da baseline pode depender dos seguintes componentes:

- .NET SDK compatível com a solução;
- Docker Desktop para os serviços locais;
- PowerShell para scripts de setup, runtime e evidência;
- Node.js e npm para a interface web;
- Git para clonagem e consulta da versão do repositório.

5.2 Setup local

A instrução principal para preparar a baseline local é:

```
docs/local-baseline-setup.md
```

Antes de executar comandos isolados, deve ser lido esse documento, porque ele concentra a sequência esperada de preparação do ambiente.

5.3 Build e testes .NET

Para validar a solução .NET, os comandos base são:

```
dotnet restore
dotnet build .\NatureProtector.sln --nologo -v minimal --configfile NuGet.Config
dotnet test .\NatureProtector.sln --no-restore --nologo -v minimal -m:1
```

O build valida a compilação dos projetos. Os testes validam o comportamento do domínio, pipeline, infraestrutura, API, simulador e componentes compartilhados.

5.4 Interface web

A interface web encontra-se em webUI/. Para validar a compilação da UI:

```
cd webUI
npm install
npm run build
```

Durante desenvolvimento, a UI é normalmente servida pelo Vite e comunica com a Backoffice API através do proxy configurado em webUI/vite.config.ts. O arranque integrado da runtime deve ser feito pelos scripts de desenvolvimento indicados no setup.

5.5 Infraestrutura local

Os serviços locais são definidos por docker-compose.yml e apoiados por scripts em infra/scripts/. Em uso normal, deve ser seguido o procedimento descrito em docs/local-baseline-setup.md. Quando aplicável, a infraestrutura pode ser iniciada por script próprio, por exemplo:

```
.\infra\scripts\up.ps1
```

Os serviços locais podem incluir PostgreSQL, RabbitMQ, InfluxDB e Grafana.

5.6 Runtime de desenvolvimento

O arranque integrado da API, Prevention Host e UI pode ser feito por script de desenvolvimento, quando disponível:

```
.\scripts\dev\start-local-runtime.ps1 -OpenBrowser -ForceRestart
```

Este script pressupõe que as dependências Docker estão disponíveis e que as portas usadas pela API e pela UI estão livres. O script gera evidência e sumários em docs/evidence/dev-runtime/, quando configurado para isso.

5.7 Execução de cenários e evidência

Os cenários e smoke tests são apoiados por scripts em `scripts/scenarios/` e `scripts/evidence/`. Um exemplo de validação B/C pode ser executado em modo de simulação do próprio script:

```
.\scripts\evidence\run-v1-bc-smoke.ps1 -DryRun
```

Quando a runtime local estiver ativa, a execução real pode apontar para a API:

```
.\scripts\evidence\run-v1-bc-smoke.ps1 -ApiBaseUrl http://localhost:5254
```

A evidência gerada deve ser consultada em `docs/evidence/`, em especial nas subpastas de `runs`, `diagnostics` e `dev-runtime`.

5.8 Coverage

A cobertura de testes pode ser gerada pelo script próprio, quando disponível:

```
.\scripts\tests\generate-coverage-report.ps1
```

O output típico é guardado em `coveragereport_core/`. Esta pasta é um artefacto gerado e não deve ser confundida com código fonte.